

项目概述

这段Go语言代码实现了一个二元逻辑回归分类器的可视化程序，使用ebiten游戏引擎展示分类过程。主要功能包括：

- 生成二维平面上的分类数据点，围绕一条真实直线分布
- 使用梯度下降算法优化逻辑回归模型参数（a、b、c）
- 实时可视化展示分类边界的拟合过程，包括样本点、真实边界和拟合边界
- 显示训练进度、交叉熵损失值和模型参数的变化

该项目非常适合理解逻辑回归和分类问题的工作原理，通过可视化可以直观地看到模型如何学习区分两个类别的决策边界。

代码结构

主要组成部分

常量定义

屏幕尺寸、坐标轴范围、数据参数等配置

全局变量

数据存储、模型参数、颜色定义等全局状态

数据生成函数

生成符合线性边界的二元分类数据

模型训练函数

计算逻辑斯蒂函数、交叉熵损失、梯度下降更新参数

可视化函数

绘制网格、坐标轴、数据点、分类边界等

游戏循环

Ebiten框架的Update和Draw方法，控制训练和渲染

程序执行流程

1. 初始化：设置随机种子、生成分类数据点、初始化模型参数
2. 游戏循环开始：
 - Update()：执行梯度下降步骤，更新模型参数
 - Draw()：绘制当前状态的所有元素，包括数据点和边界
3. 训练完成：模型参数收敛，拟合边界接近真实边界

核心算法

逻辑回归模型

逻辑回归模型通过逻辑斯蒂函数（sigmoid）将线性组合映射到[0,1]区间的概率值：

$$p = 1 / (1 + e^{-(a + b \cdot x + c \cdot y)})$$

其中：

- p 是属于类别1的概率
- x, y 是输入特征（二维坐标）
- a 是偏置参数
- b, c 是特征的权重参数

代码实现：

```
func logistic(a, b, c, x, y float64) float64 {  
    g := a + b*x + c*y  
    return 1.0 / (1.0 + math.Exp(-g))  
}
```

交叉熵损失函数

使用交叉熵损失函数衡量分类效果：

$$L = -(1/n) \cdot \sum (y_i \cdot \log(p_i) + (1-y_i) \cdot \log(1-p_i))$$

代码实现：

```
func crossEntropyLoss(yTrue []int, xData, yData []float64, a, b, c float64) float64 {  
    m := len(yTrue)  
    sumLoss := 0.0  
    for i := range yTrue {  
        p := logistic(a, b, c, xData[i], yData[i])  
        if yTrue[i] == 1 {  
            sumLoss -= math.Log(p)  
        } else {  
            sumLoss -= math.Log(1 - p)  
        }  
    }  
    return sumLoss / float64(m)  
}
```

梯度下降

通过计算损失函数对各参数的梯度，沿梯度反方向更新参数：

$$a = a - lr \cdot \partial L / \partial a$$

$$b = b - lr \cdot \partial L / \partial b$$

$$c = c - lr \cdot \partial L / \partial c$$

梯度计算与参数更新的代码实现：

```
func (g *Game) Update() error {  
    if currentIter < totalIterations {  
        gradA := gradientA(labels, xData, yData, a, b, c)
```

```

        gradB := gradientB(labels, xData, yData, a, b, c)
        gradC := gradientC(labels, xData, yData, a, b, c)

        a -= learningRate * gradA
        b -= learningRate * gradB
        c -= learningRate * gradC

        loss = crossEntropyLoss(labels, xData, yData, a, b, c)
        currentIter++
    }
    return nil
}

```

分类边界

当 $p=0.5$ 时，得到分类边界： $a + b \cdot x + c \cdot y = 0$ ，即 $y = (-a - b \cdot x) / c$ 。随着训练进行，模型学习到的边界会逐渐接近真实边界。

可视化部分

绘制元素

坐标系与网格

通过drawGrid()和drawAxis()函数实现：

- 绘制x轴和y轴，带刻度标识
- 添加灰色网格线，便于观察数据位置
- 标注坐标轴刻度值

数据点与边界

通过drawDataPoints()和drawDecisionBoundary()函数实现：

- 用蓝色和红色绘制两类数据点
- 用绿色绘制真实分类边界
- 用橙色绘制当前拟合的分类边界

状态信息展示

通过多个辅助函数展示训练状态：

```

// 绘制方程文本
func drawEquations(screen *ebiten.Image) { ... }

// 绘制训练信息
func drawTrainingInfo(screen *ebiten.Image) { ... }

```

展示的关键信息包括：

- 当前迭代次数和总迭代次数
- 当前交叉熵损失值
- 当前模型参数a、b、c
- 真实边界和拟合边界的方程

坐标转换

数学坐标到屏幕坐标的映射：

```
// 坐标转换：数学坐标转屏幕坐标
func mathToScreen(x, y float64) (float64, float64) {
    screenX := (x - xMinPlot) / (xMaxPlot - xMinPlot) * float64(screenWidth)
    screenY := (1 - (y - yMinPlot)/(yMaxPlot - yMinPlot)) * float64(screenHeight)
    return screenX, screenY
}
```

该函数确保数学坐标系中的点能正确显示在屏幕上，考虑了不同的坐标方向和范围。

参数说明

关键参数

参数	含义	默认值	调整建议
numPoints	数据点数量	2000	数据越多，模型越稳定，但计算量越大
learningRate	学习率	0.006	过大会导致不收敛，过小会导致收敛慢
totalIterations	总迭代次数	200000	足够大以确保收敛，可根据损失变化调整
noiseMax	噪声最大值	1.5	值越大，数据越分散，分类越困难
trueSlope	真实边界斜率	1.342	定义真实分类边界的斜率
trueIntercept	真实边界截距	2.45	定义真实分类边界的截距